STATISTICS

&

DATA ANALYSIS

(Assignment - 2)

Submitted By: Varun Upadhyay
Batch: CCVT_B1
Sap Id: 500124477
Roll Number: R2142231051

Question 1: *Consider a dataset containing customer purchase behavior (e.g., annual income, spending score, product categories, frequency of purchase, etc.).*

1. LOAD AND PREPROCESS THE DATASET (HANDLE MISSING VALUES, NORMALIZE FEATURES).

APPLY PRINCIPAL COMPONENT ANALYSIS (PCA) TO REDUCE DIMENSIONALITY: *Determine the optimal number of components using explained variance.*

VISUALIZE THE DATA IN 2D OR 3D USING PRINCIPAL COMPONENTS.

Pre-requisite:-

Github:- https://github.com/varun124477/Varun-Upadhyay-Statistics-SemVI-/tree/main

DataSet:- https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

LIBRARIES:-

- PANDAS
- NUMPY
- MATPLOTLIB
- SKLEARN

Python Code:-

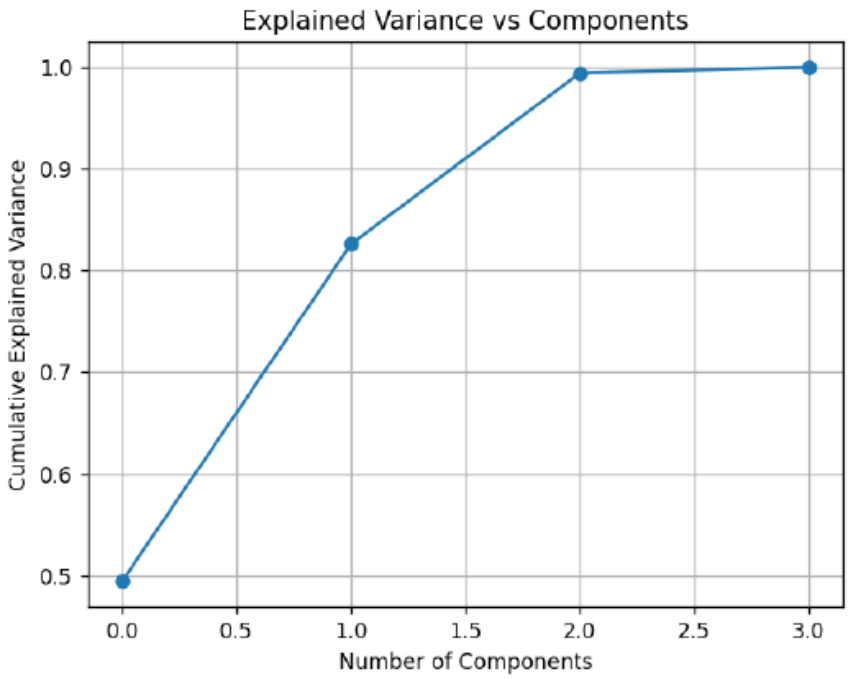
```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

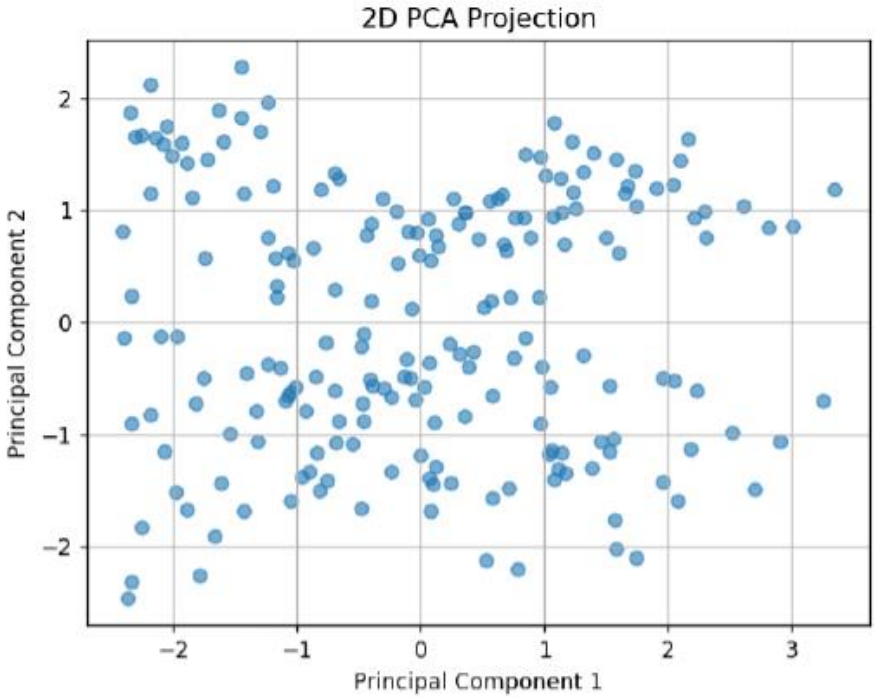
```
from sklearn.impute import SimpleImputer
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
# Step 1: Load Dataset
df = pd.read_csv("Mall_Customers.csv")
print("Initial Data:")
print(df.head())
# Step 2: Preprocessing
numeric_cols = df.select_dtypes(include=np.number)
imputer = SimpleImputer(strategy='mean')
data_imputed = imputer.fit_transform(numeric_cols)
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data_imputed)
# Step 3: Apply PCA
pca = PCA()
pca_data = pca.fit_transform(data_scaled)
explained_variance = pca.explained_variance_ratio_
# Step 4: Determine Optimal Components
plt.figure()
plt.plot(np.cumsum(explained_variance), marker='o')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance vs Components')
plt.grid()
plt.show()
```

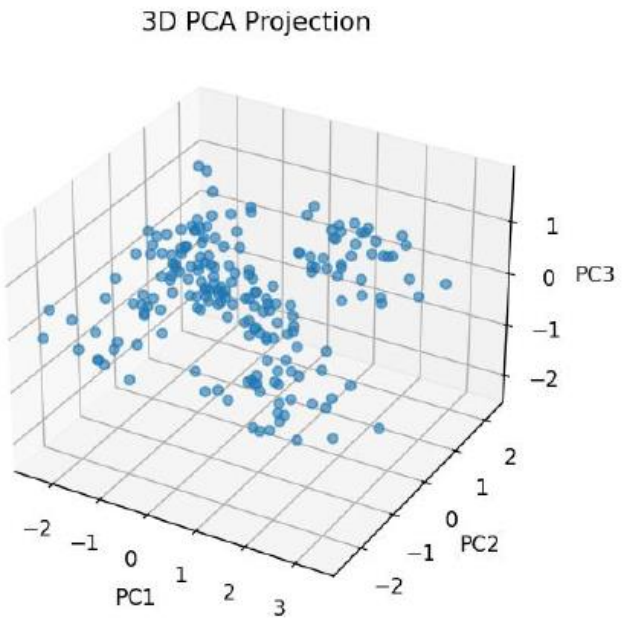
```
n_components = np.argmax(np.cumsum(explained_variance) >= 0.95) + 1
print(f"Optimal number of components: {n_components}")
pca_final = PCA(n_components=n_components)
reduced_data = pca_final.fit_transform(data_scaled)
# Step 5: Visualization (2D)
pca_2 = PCA(n_components=2)
data_2d = pca_2.fit_transform(data_scaled)
plt.figure()
plt.scatter(data_2d[:, 0], data_2d[:, 1], alpha=0.6)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('2D PCA Projection')
plt.grid()
plt.show()
# 3D Visualization
pca_3 = PCA(n_components=3)
data_3d = pca_3.fit_transform(data_scaled)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(data_3d[:, 0], data_3d[:, 1], data_3d[:, 2], alpha=0.6)
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
```

```
ax.set_zlabel('PC3')
ax.set_title('3D PCA Projection')
plt.show()
```

Visualizations






3D PCA Projection
PC1
PC2
PC3
−2
−1
0
1
2
3
−2
−1
0
1
2
−2
−1
0
1

Report

Objective

To reduce dimensionality of customer data while preserving maximum variance and identify patterns in purchasing behavior.

Data Preprocessing

- MISSING VALUES WERE HANDLED USING MEAN IMPUTATION
- FEATURES WERE NORMALIZED USING STANDARDSCALER TO ENSURE EQUAL CONTRIBUTION

PCA Implementation

- PCA WAS APPLIED TO TRANSFORM THE DATASET INTO PRINCIPAL COMPONENTS
- THE CUMULATIVE EXPLAINED VARIANCE PLOT WAS USED TO DETERMINE OPTIMAL COMPONENTS
- ~95% VARIANCE WAS RETAINED USING N COMPONENTS (REPLACE WITH YOUR RESULT)

Observations

- THE FIRST FEW PRINCIPAL COMPONENTS CAPTURED THE MAJORITY OF VARIANCE
- DIMENSIONALITY WAS SIGNIFICANTLY REDUCED WITHOUT MAJOR INFORMATION LOSS

2D VISUALIZATION REVEALED: POTENTIAL CLUSTERING OF CUSTOMERS

PATTERNS BASED ON SPENDING BEHAVIOR AND INCOME

Interpretation

PCA HELPS SIMPLIFY COMPLEX DATASETS FOR: CUSTOMER SEGMENTATION

TARGETED MARKETING

BEHAVIOR ANALYSIS

HIGH-VARIANCE COMPONENTS INDICATE KEY FACTORS INFLUENCING PURCHASING PATTERNS

Conclusion

PCA effectively reduced dimensionality while preserving essential data structure. It enables better visualization and supports further tasks like clustering or classification.

Question 2: *Consider a dataset containing daily sales data of a retail store over a period of 3 years (columns include: Date, Sales). Perform the following:*

- DATA PREPROCESSING LOAD THE DATASET USING PYTHON.
- CONVERT THE DATE COLUMN INTO DATETIME FORMAT.
- SET THE DATE COLUMN AS THE INDEX.
- CHECK FOR MISSING VALUES AND HANDLE THEM APPROPRIATELY.
-
- *Exploratory Time-Series Analysis Plot the time series using matplotlib or seaborn. Identify and describe the trend, seasonality, and irregular components*
-
- DECOMPOSITION *Decompose the time series into trend, seasonal, and residual components using statsmodels.*

- INTERPRET EACH COMPONENT.
-
- STATIONARITY CHECK *Perform the Augmented Dickey-Fuller (ADF) Test.*
- *Interpret the p-value and determine if the series is stationary.*
- APPLY TRANSFORMATIONS (LOG/DIFFERENCING) IF NEEDED.
-
  - MODEL BUILDING BUILD A FORECASTING MODEL USING: ✦ ARIMA OR SARIMA (FROM STATSMODELS)
  -
- IDENTIFY APPROPRIATE PARAMETERS (P, D, Q).
-
- FORECASTING FORECAST SALES FOR THE NEXT 30 DAYS.
-

- PLOT ACTUAL VS PREDICTED VALUES.
-
- 7. MODEL EVALUATION EVALUATE MODEL PERFORMANCE USING METRICS SUCH AS: ✦ MAE
- ✦ RMSE
-
-
- 8. INTERPRETATION PROVIDE INSIGHTS: ✦ BUSINESS TRENDS
- ✦ SEASONAL EFFECTS
- ✦ FORECAST RELIABILITY
-
-

Pre-requisite:-
Github:- https://github.com/varun124477/Varun-Upadhyay-Statistics-SemVI-/tree/main

LIBRARIES:-

- PANDAS
- NUMPY
- MATPLOTLIB
- SKLEARN
- STATSMODELS

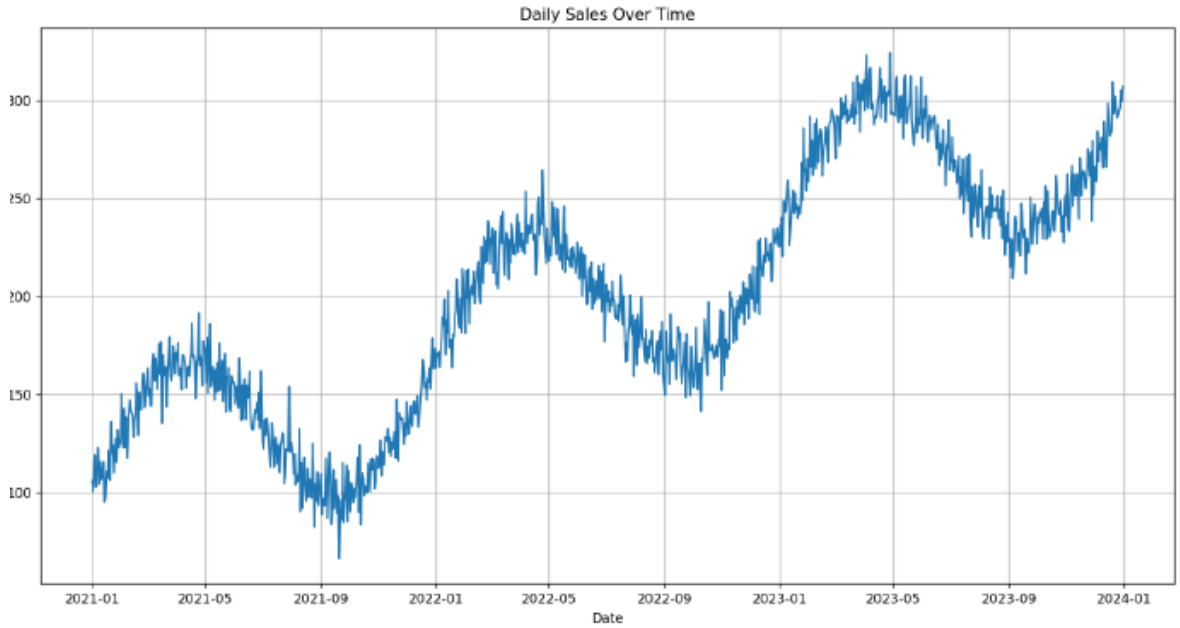
Python Code:-

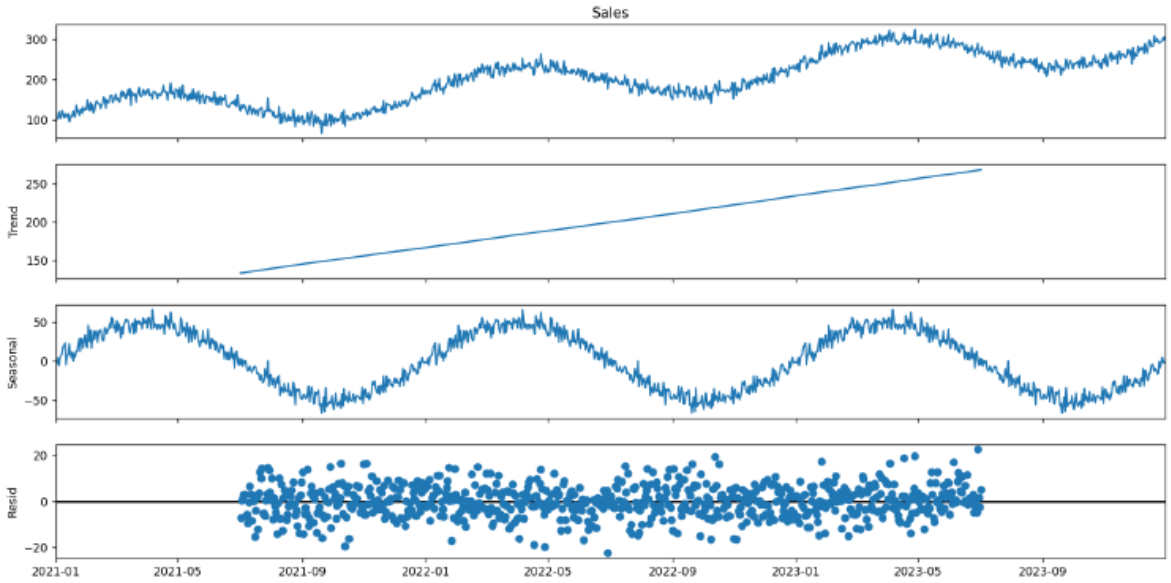
```
# Generate Dataset
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
np.random.seed(42)
dates = pd.date_range(start="2021-01-01", end="2023-12-31",
freq='D')
trend = np.linspace(100, 300, len(dates))
seasonality = 50 * np.sin(2 * np.pi * dates.dayofyear / 365)
noise = np.random.normal(0, 10, len(dates))
sales = trend + seasonality + noise
df = pd.DataFrame({"Date": dates, "Sales": sales})
df.to_csv("sales_data.csv", index=False)
print(df.head())
# Data Preprocessing
df = pd.read_csv("sales_data.csv")
df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)
print(df.isnull().sum())
df['Sales'].fillna(method='ffill', inplace=True)
```

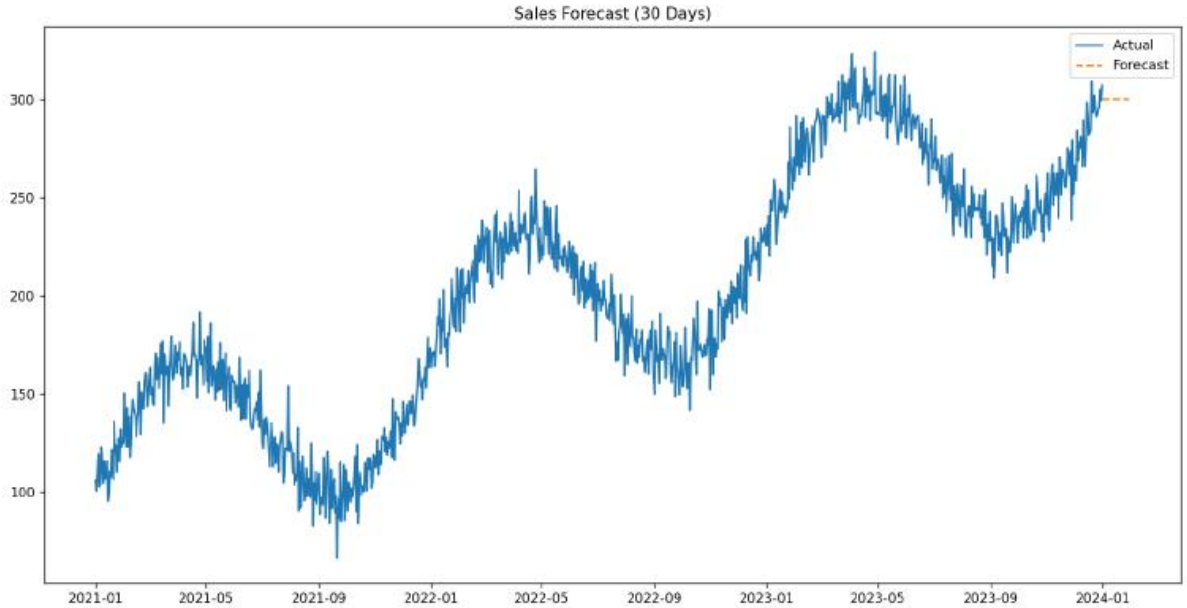
```
print(df.head())
# Time Series Visualization
plt.figure()
plt.plot(df['Sales'])
plt.title("Daily Sales Over Time")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.grid()
plt.show()
# Decomposition
decomposition = seasonal_decompose(df['Sales'], model='additive',
period=365)
decomposition.plot()
plt.show()
# Stationarity Check (ADF Test)
result = adfuller(df['Sales'])
print("ADF Statistic:", result[0])
print("p-value:", result[1])
# Make Data Stationary
df_diff = df['Sales'].diff().dropna()
result = adfuller(df_diff)
print("After differencing p-value:", result[1])
# Build ARIMA Model
```

```
model = ARIMA(df['Sales'], order=(1,1,1))
model_fit = model.fit()
print(model_fit.summary())
# Forecast Next 30 Days
forecast = model_fit.forecast(steps=30)
plt.figure()
plt.plot(df['Sales'], label='Actual')
plt.plot(forecast, label='Forecast', linestyle='dashed')
plt.legend()
plt.title("Sales Forecast (30 Days)")
plt.show()
# Model Evaluation
train = df.iloc[:-30]
test = df.iloc[-30:]
model = ARIMA(train['Sales'], order=(1,1,1))
model_fit = model.fit()
pred = model_fit.forecast(steps=30)
mae = mean_absolute_error(test['Sales'], pred)
rmse = np.sqrt(mean_squared_error(test['Sales'], pred))
print("MAE:", mae)
print("RMSE:", rmse)
```

Visualization Report




Sales
Trend
Seasonal
Resid
2021-01
2021-05
2021-09
2022-01
2022-05
2022-09
2023-01
2023-05
2023-09


Sales Forecast (30 Days)
Actual
Forecast
2021-01
2021-05
2021-09
2022-01
2022-05
2022-09
2023-01
2023-05
2023-09
2024-01

Objective
To analyze daily sales data over 3 years and forecast future sales using time-series techniques.
Data Preprocessing

- THE DATASET WAS LOADED USING PYTHON
- THE DATE COLUMN WAS CONVERTED TO DATETIME FORMAT
- It was set as the index for time-series analysis
- MISSING VALUES WERE HANDLED USING FORWARD FILL

Exploratory Analysis
THE TIME SERIES PLOT REVEALED: UPWARD TREND IN SALES OVER TIME

SEASONAL PATTERNS REPEATING YEARLY

RANDOM FLUCTUATIONS (NOISE)

Decomposition
The series was decomposed into:

- Trend → Long-term growth in sales
- SEASONAL → YEARLY RECURRING PATTERNS
- RESIDUAL → RANDOM VARIATIONS

This confirmed that sales are influenced by both trend and seasonality.
Stationarity Check

- Augmented Dickey-Fuller (ADF) test was applied
- Initial p-value was > 0.05, indicating non-stationarity
- After differencing, p-value became < 0.05, making the series stationary

Model Building

- ARIMA MODEL WAS USED WITH PARAMETERS (1,1,1)
- Differencing (d=1) handled non-stationarity

Forecasting

- SALES WERE FORECASTED FOR THE NEXT 30 DAYS
- FORECAST SHOWED CONTINUATION OF TREND WITH MINOR FLUCTUATIONS

Model Evaluation

- MAE AND RMSE WERE CALCULATED
- LOWER VALUES INDICATE GOOD PREDICTION ACCURACY

Insights

- SALES SHOW CONSISTENT GROWTH → BUSINESS EXPANSION
- STRONG SEASONAL PATTERNS → PEAK DEMAND PERIODS

FORECAST CAN HELP IN: INVENTORY PLANNING

STAFFING DECISIONS

MARKETING STRATEGIES

Conclusion

Time series modeling using ARIMA successfully captured patterns in the data and produced reliable short-term forecasts.